

Laravel ও Backend Infrastructure

সিনিয়র লেভেল রিভিশন — ১৫ মিনিট। প্রতিটা প্রস্নে Description পড়ে Key Point-গুলো একবার চোখ বুলালেই যথেষ্ট।

১. Laravel-এর Request Lifecycle কীভাবে কাজ করে?

Description: একটা HTTP request ঢোকা থেকে response বের হওয়া পর্যন্ত Laravel ভেতরে কী কী ধাপ ঘটে — সিনিয়রদের কাছে এটা জিজ্ঞেস করা হয় ফ্রেমওয়ার্কের ভেতরটা বোঝেন কিনা যাচাই করতে।

৬টা Key Point:

1. Entry point `public/index.php` → Composer autoload → `bootstrap/app.php` থেকে Application instance তৈরি হয়
2. HTTP Kernel request-টা handle করে — প্রথমে global middleware stack চালায় (TrimStrings, TrustProxies ইত্যাদি)
3. Service Provider-গুলোর `register()` আগে, তারপর সবগুলোর `boot()` চলে — এই order-টা গুরুত্বপূর্ণ
4. Router request-টা match করে route middleware group (web/api) চালিয়ে controller-এ পাঠায়
5. Controller-এর dependency-গুলো Service Container automatic resolve করে (method injection)
6. Response object আবার middleware-এর ভেতর দিয়ে উল্টো পথে ফিরে client-এ যায়, শেষে `terminate()` চলে

২. Service Container ও Service Provider-এর পার্থক্য ও ব্যবহার

Description: Dependency Injection-এর মূল ইঞ্জিন হলো Container, আর Provider হলো সেখানে জিনিস bind করার জায়গা। Interface-to-implementation swap করার প্র্যাকটিক্যাল অভিজ্ঞতা আছে কিনা সেটাই দেখা হয়।

৬টা Key Point:

1. Container হলো class dependency resolve করার IoC engine; Provider হলো bootstrap/binding রেজিস্টার করার জায়গা
2. `bind()` প্রতিবার নতুন instance দেয়, `singleton()` একবারই বানিয়ে reuse করে
3. Interface bind করলে (`bind(PaymentGateway::class, BkashGateway::class)`) implementation swap করা যায় কোড না বদলে — টেস্টও mock করা সহজ
4. Contextual binding: একই interface-এর ভিন্ন implementation ভিন্ন class-এ inject করা যায় (`when()->needs()->give()`)
5. `register()`-এ শুধু binding, `boot()`-এ অন্য service ব্যবহার — কারণ boot-এর সময় সব provider registered থাকে
6. Deferred provider ব্যবহার করলে service দরকার না হওয়া পর্যন্ত load হয় না — boot performance বাড়ে

৩. Eloquent-এর N+1 প্রবলেম ও Query Optimization

Description: প্রোডাকশনে সবচেয়ে কমন performance বাগ। কীভাবে ধরা যায় ও ঠিক করা যায় — সিনিয়র হিসেবে হাতেকলমে অভিজ্ঞতা আশা করা হয়।

৬টা Key Point:

1. Loop-এর ভেতর relationship access করলে প্রতি iteration-এ আলাদা query চলে — এটাই N+1
2. সমাধান: `with()` দিয়ে eager loading; nested হলে `with('orders.items')`

3. `withCount()`, `withSum()` দিয়ে aggregate আনা — পুরো relation load না করে
4. `select()` দিয়ে দরকারি column আনা, বড় টেবিলে `chunk()/cursor()/lazy()` ব্যবহার করা
5. Debug: Laravel Debugbar, Telescope, বা `Model::preventLazyLoading()` (dev-এ exception ছোড়ে)
6. Index না থাকলে eager loading-ও slow — **EXPLAIN** দিয়ে query plan দেখা অভ্যাস করা

8. Queue ও Job কীভাবে ডিজাইন করা উচিত?

Description: Heavy কাজ (email, report, import) request cycle থেকে সরিয়ে background-এ নেওয়া। Failure handling সহ প্রোডাকশন-গ্রেড queue ডিজাইনের অভিজ্ঞতা যাচাই হয়।

৬টা Key Point:

1. Driver choice: Redis (fast, সাধারণ ক্ষেত্রে যথেষ্ট), SQS (managed, scale), database (ছোট প্রজেক্ট/সহজ debug)
2. Job idempotent বানানো জরুরি — retry হলে যেন দুইবার effect না হয় (unique check বা `ShouldBeUnique`)
3. `$tries`, `$backoff`, `retryUntil()` দিয়ে retry policy; ব্যর্থ হলে `failed()` মেথডে cleanup/notify
4. `failed_jobs` টেবিল মনিটর করা + `queue:retry` দিয়ে পুনরায় চালানো; Horizon দিয়ে dashboard/metrics
5. Job-এ পুরো model পাঠালে `SerializesModels` শুধু ID রাখে — job চলার সময় fresh data আসে (stale data সমস্যাও এড়ায়)
6. Chain (`Bus::chain`) sequential-এর জন্য, Batch (`Bus::batch`) parallel + progress tracking-এর জন্য

৫. Middleware কখন ও কীভাবে ব্যবহার করবেন?

Description: Request/response pipeline-এ cross-cutting concern (auth, tenant switch, logging) বসানোর জায়গা। কোন logic middleware-এ যাবে আর কোনটা যাবে না, সেই judgment দেখা হয়।

৬টা Key Point:

1. Middleware-এ শুধু cross-cutting concern: auth, rate limit, CORS, locale, tenant resolution — business logic নয়
2. Before middleware `$next($request)`-এর আগে কাজ করে, after middleware response পাওয়ার পরে
3. Route middleware-এ parameter পাঠানো যায়: `->middleware('permission:VIEW_REPORT')`
4. Global vs group vs route-specific — যত ছোট scope-এ রাখা যায় তত ভালো
5. `terminate()` মেথডে response পাঠানোর পরের কাজ করা যায় (logging, metrics) — user wait করে না
6. Multi-tenant অ্যাপে middleware-ই tenant DB connection switch করার সঠিক জায়গা (আমাদের Sokrio-তেও `tenant` middleware তাই করে)

৬. Database Transaction ও Race Condition হ্যান্ডলিং

Description: টাকা/স্টক জড়িত multi-step write-এ data consistency রক্ষা করা। Deadlock ও concurrent update-এর বাস্তব অভিজ্ঞতা জিজ্ঞেস করা হয়।

৬টা Key Point:

1. Multi-step write সবসময় `DB::transaction(fn () => ...)` এ — কোনো একটা fail করলে সব rollback
2. Pessimistic lock: `lockForUpdate()` — স্টক কমানোর মতো ক্ষেত্রে দুই request একসাথে পড়ে দুইবার কমানো ঠেকায়
3. Optimistic lock: version column চেক করে update — conflict কম হলে বেশি efficient
4. Deadlock হলে transaction retry: `DB::transaction($callback, attempts: 3)`
5. Transaction-এর ভেতর external call (API, mail) নয় — rollback হলে সেগুলো ফেরানো যায় না; `DB::afterCommit()` বা `afterCommit` job ব্যবহার

6. Atomic increment (`increment()`) ছোট ক্ষেত্রে lock ছাড়াই safe — race এড়ানোর সবচেয়ে সম্ভাব্য উপায়

৭. API Authentication: Sanctum vs Passport, কোনটা কখন?

Description: Token-based auth-এর দুই অপশনের trade-off। বেশিরভাগ প্রজেক্টে Sanctum যথেষ্ট — কেন, সেটা ব্যাখ্যা করতে পারা জরুরি।

৬টা Key Point:

1. Sanctum: lightweight, personal access token + SPA cookie auth — first-party app-এর জন্য যথেষ্ট
2. Passport: full OAuth2 server (authorization code, client credentials) — third-party integration লাগলে তবেই
3. SPA-তে Sanctum cookie mode: CSRF protected, token localStorage-এ রাখার XSS ঝুঁকি নেই
4. Mobile app-এ Sanctum plain bearer token — `createToken('device-name', ['abilities'])`
5. Token abilities দিয়ে coarse scope, আর fine-grained কাজে permission/gate ব্যবহার
6. Token revoke/expiry স্ট্র্যাটেজি রাখা: logout-এ token delete, `expiration` config, প্রয়োজনে refresh flow

৮. Event, Listener, Observer — Decoupling কৌশল

Description: "Order create হলে SMS যাবে, স্টক কমবে, log হবে" — এই side effect-গুলো controller-এ না লিখে কীভাবে decouple করবেন সেটার প্রশ্ন।

৬টা Key Point:

1. Event fire করে দিলে একাধিক Listener আলাদাভাবে react করে — মূল flow পরিষ্কার থাকে
2. Listener-এ `ShouldQueue` লাগালে async চলে — slow কাজ (mail/SMS) request block করে না
3. Observer model lifecycle hook-এর জন্য (created, updated, deleted) — audit log-এর কমন প্যাটার্ন
4. সতর্কতা: বেশি observer/event হলে flow অদৃশ্য হয়ে debug কঠিন — গুরুত্বপূর্ণ business flow explicit রাখা ভালো
5. `DB::afterCommit / ShouldHandleEventsAfterCommit` — transaction rollback হলে event-এর কাজ যেন না ঘটে
6. Event vs Job পার্থক্য: Event = "কিছু ঘটেছে" (একাধিক শ্রোতা), Job = "এই কাজটা করো" (একটা executor)

৯. Multi-tenant Architecture Laravel-এ কীভাবে করবেন?

Description: SaaS প্রোডাক্টে প্রতিটা client-এর ডেটা আলাদা রাখার কৌশল। Database-per-tenant vs shared-database trade-off — আমার দৈনন্দিন কাজের এরিয়া, তাই confident উত্তর দেওয়া যাবে।

৬টা Key Point:

1. দুই মূল প্যাটার্ন: single DB + `tenant_id` column (সহজ, ঝুঁকি: scope মিস) vs DB-per-tenant (শক্ত isolation, migration overhead)
2. Landlord DB-তে platform data (org, billing), tenant DB-তে business data — Sokrio-তে এই প্যাটার্নই চলে
3. Middleware-এ tenant resolve (domain/header থেকে) করে connection reconnect — পরে অবশ্যই landlord-এ switch back
4. Shared-DB হলে Global Scope দিয়ে সব query-তে `tenant_id` auto-apply — ম্যানুয়াল where-এর ভরসায় থাকা বিপজ্জনক
5. Cache/queue key-তেও tenant scope রাখা বাধ্যতামূলক — নাহলে এক tenant-এর report আরেকজন দেখে ফেলে
6. Migration সব tenant-এ লুপ করে চালাতে হয় — deploy script-এ এটা automate করা, ব্যর্থ tenant track করা

১০. Laravel অ্যাপ্লিকেশন Scale ও Performance Optimization

Description: ট্রাফিক বাড়লে কোথায় কী optimize করবেন — caching থেকে horizontal scaling পর্যন্ত একটা লেয়ারড উত্তর আশা করা হয়।

৬টা Key Point:

1. প্রথম ধাপ সবসময় measure: slow query log, Telescope/APM — অনুমানে optimize নয়
2. `config:cache`, `route:cache`, `event:cache`, OPcache, composer `--optimize-autoloader` — ফ্রি performance
3. Heavy read cache করা: `Cache::remember($key, $ttl, ...)` — key-তে scope (tenant/territory) রাখা, invalidation প্ল্যান করা
4. Read-heavy হলে DB read replica (`read/write` connection config) — replication lag মাথায় রাখা
5. Horizontal scale: stateless app server + LB; session/cache Redis-এ, file S3-তে — sticky session-এর দরকার যেন না হয়
6. Octane (Swoole/RoadRunner) দিয়ে boot cost বাদ — তবে memory leak/static state-এ সাবধান থাকতে হয়